Proving Data Structures Correct: ADTs & 2-3 Trees

Computer-Assisted Theorem Proving (7CCSACTP)

Mohammad Abdulaziz

Abstract Data Types

A methodological interlude:

©Mohammad Abdulaziz

Abstract Data Types

A methodological interlude:

A closer look at ADT principles
and their realisation in Isabelle

©Mohammad Abdulaziz

Abstract Data Types

A methodological interlude:

A closer look at ADT principles
and their realisation in Isabelle

Set and binary search tree as examples
(ignoring `delete`)

©Mohammad Abdulaziz

Defining ADTs

ADT = *interface* + *specification*

©Mohammad Abdulaziz

Example (Set interface)

```
empty :: 's
insert :: 'a ⇒ 's ⇒ 's
isin :: 's ⇒ 'a ⇒ bool
```

©Mohammad Abdulaziz

Example (Set interface)

```
empty :: 's
insert :: 'a ⇒ 's ⇒ 's
isin :: 's ⇒ 'a ⇒ bool
```

We assume that each ADT describes one

Type of Interest T

Example (Set interface)

```
empty :: 's
insert :: 'a ⇒ 's ⇒ 's
isin :: 's ⇒ 'a ⇒ bool
```

We assume that each ADT describes one

Type of Interest T

Above: $T =$'s

Model-oriented specification

Specify type T via a model = existing HOL type A

©Mohammad Abdulaziz

Model-oriented specification

Specify type T via a model = existing HOL type A

Motto: T should behave like A

©Mohammad Abdulaziz

Model-oriented specification

Specify type T via a model = existing HOL type A

Motto: T should behave like A

Specification of "behaves like" via an

- *abstraction function* $\alpha :: T \Rightarrow A$

©Mohammad Abdulaziz

Model-oriented specification

Specify type T via a model = existing HOL type A

Motto: T should behave like A

Specification of "behaves like" via an

- *abstraction function* $\alpha :: T \Rightarrow A$

Only some elements of T represent elements of A:

- *invariant* invar $:: T \Rightarrow$ bool

©Mohammad Abdulaziz

Model-oriented specification

Specify type T via a model = existing HOL type A

Motto: T should behave like A

Specification of "behaves like" via an

- *abstraction function* $\alpha :: T \Rightarrow A$

Only some elements of T represent elements of A:

- *invariant* invar :: $T \Rightarrow$ bool

α and invar are part of the interface,
but only for specification and verification purposes

©Mohammad Abdulaziz

Example (Set ADT)

```
empty :: ...
insert :: ...
isin :: ...
```

©Mohammad Abdulaziz

Example (Set ADT)

```
empty :: ...
insert :: ...
isin :: ...
set ::    's ⇒ 'a set
```

©Mohammad Abdulaziz

Example (Set ADT)

```
empty :: ...
insert :: ...
isin :: ...
set ::    's ⇒ 'a set
```

(name arbitrary)

Example (Set ADT)

```
empty :: ...
insert :: ...
isin :: ...
set ::    's ⇒ 'a set   (name arbitrary)
invar ::  's ⇒ bool
```

Example (Set ADT)

```
empty :: ...
insert :: ...
isin :: ...
set ::    's ⇒ 'a set   (name arbitrary)
invar ::  's ⇒ bool      (name arbitrary)
```

Example (Set ADT)

```
empty :: ...
insert :: ...
isin :: ...
set ::    's ⇒ 'a set    (name arbitrary)
invar ::  's ⇒ bool      (name arbitrary)

              set empty = {}
```

©Mohammad Abdulaziz

Example (Set ADT)

```
empty :: ...
insert :: ...
isin :: ...
set ::    's ⇒ 'a set   (name arbitrary)
invar ::  's ⇒ bool     (name arbitrary)

              set empty = {}
              set(insert x s) = set s ∪ {x}
```

Example (Set ADT)

```
empty :: ...
insert :: ...
isin :: ...
set ::    's ⇒ 'a set   (name arbitrary)
invar ::  's ⇒ bool     (name arbitrary)
```

```
set empty = {}
set(insert x s) = set s ∪ {x}
isin s x = (x ∈ set s)
```

Example (Set ADT)

```
empty :: ...
insert :: ...
isin :: ...
set ::    's ⇒ 'a set   (name arbitrary)
invar ::  's ⇒ bool     (name arbitrary)

                set empty = {}
 invar s ⟹  set(insert x s) = set s ∪ {x}
 invar s ⟹  isin s x = (x ∈ set s)
```

Example (Set ADT)

```
empty :: ...
insert :: ...
isin :: ...
set ::    's ⇒ 'a set   (name arbitrary)
invar ::  's ⇒ bool     (name arbitrary)

                set empty = {}
 invar s ⟹      set(insert x s) = set s ∪ {x}
 invar s ⟹      isin s x = (x ∈ set s)
                invar empty
```

Example (Set ADT)

```
empty :: ...
insert :: ...
isin :: ...
set ::    's ⇒ 'a set   (name arbitrary)
invar ::  's ⇒ bool     (name arbitrary)

                set empty = {}
 invar s ⟹      set(insert x s) = set s ∪ {x}
 invar s ⟹      isin s x = (x ∈ set s)
                invar empty
 invar s ⟹      invar(insert x s)
```

In Isabelle: **locale**

locale `Set =`

In Isabelle: **locale**

```
locale Set =
fixes empty :: 's
fixes insert :: 'a ⇒ 's ⇒ 's
fixes isin :: 's ⇒ 'a ⇒ bool
```

In Isabelle: **locale**

```
locale Set =
fixes empty :: 's
fixes insert :: 'a ⇒ 's ⇒ 's
fixes isin :: 's ⇒ 'a ⇒ bool
fixes set :: 's ⇒ 'a set
fixes invar :: 's ⇒ bool
```

In Isabelle: **locale**

```
locale Set =
fixes empty :: 's
fixes insert :: 'a ⇒ 's ⇒ 's
fixes isin :: 's ⇒ 'a ⇒ bool
fixes set :: 's ⇒ 'a set
fixes invar :: 's ⇒ bool
assumes set empty = {}
assumes invar s ⟹ isin s x = (x ∈ set s)
assumes invar s ⟹ set(insert x s) = set s ∪ {x}
```

In Isabelle: **locale**

```
locale Set =
fixes empty :: 's
fixes insert :: 'a ⇒ 's ⇒ 's
fixes isin :: 's ⇒ 'a ⇒ bool
fixes set :: 's ⇒ 'a set
fixes invar :: 's ⇒ bool
assumes set empty = {}
assumes invar s ⟹ isin s x = (x ∈ set s)
assumes invar s ⟹ set(insert x s) = set s ∪ {x}
assumes invar empty
assumes invar s ⟹ invar(insert x s)
```

©Mohammad Abdulaziz

In Isabelle: **locale**

```
locale Set =
fixes empty :: 's
fixes insert :: 'a ⇒ 's ⇒ 's
fixes isin :: 's ⇒ 'a ⇒ bool
fixes set :: 's ⇒ 'a set
fixes invar :: 's ⇒ bool
assumes set empty = {}
assumes invar s ⟹ isin s x = (x ∈ set s)
assumes invar s ⟹ set(insert x s) = set s ∪ {x}
assumes invar empty
assumes invar s ⟹ invar(insert x s)
```

See `HOL/Data_Structures/Set_Specs.thy`

Formally, in general

To ease notation, generalise α and invar (conceptually):

©Mohammad Abdulaziz

Formally, in general

To ease notation, generalise α and invar (conceptually):

α is the identity and invar is `True`
on types other than T

Formally, in general

To ease notation, generalise α and invar (conceptually):

α is the identity and invar is `True`
on types other than T

Specification of each interface function f (on T):

Formally, in general

To ease notation, generalise α and invar (conceptually):

α is the identity and invar is `True`
on types other than T

Specification of each interface function f (on T):

- f must behave like some function f_A (on A):

Formally, in general

To ease notation, generalise α and invar (conceptually):

α is the identity and invar is `True`
on types other than T

Specification of each interface function f (on T):

- f must behave like some function f_A (on A):
  $\text{invar}\ t_1 \wedge \cdots \wedge \text{invar}\ t_n \Longrightarrow$
  $\alpha(f\ t_1 \ldots t_n) = f_A(\alpha\, t_1) \ldots (\alpha\, t_n)$

©Mohammad Abdulaziz

Formally, in general

To ease notation, generalise α and invar (conceptually):

α is the identity and invar is `True`
on types other than T

Specification of each interface function f (on T):

- f must behave like some function f_A (on A):
  $\text{invar}\ t_1 \wedge \cdots \wedge \text{invar}\ t_n \Longrightarrow$
  $\alpha(f\ t_1 \ldots t_n) = f_A(\alpha\, t_1) \ldots (\alpha\, t_n)$
  (α is a homomorphism)

©Mohammad Abdulaziz

Formally, in general

To ease notation, generalise α and invar (conceptually):

α is the identity and invar is `True`
on types other than T

Specification of each interface function f (on T):

- f must behave like some function f_A (on A):
  $\text{invar}\ t_1 \wedge \cdots \wedge \text{invar}\ t_n \implies$
  $\alpha(f\ t_1 \ldots t_n) = f_A(\alpha\ t_1) \ldots (\alpha\ t_n)$

  (α is a homomorphism)
- f must preserve the invariant:
  $\text{invar}\ t_1 \wedge \cdots \wedge \text{invar}\ t_n \implies \text{invar}(f\ t_1 \ldots t_n)$

©Mohammad Abdulaziz

Using ADTs

The purpose of an ADT is to provide a context
for implementing generic algorithms
parameterised with the interface functions of the ADT.

©Mohammad Abdulaziz

Example

locale `Set =`
fixes ...
assumes ...

begin
fun `set_of_list` **where**
```
"set_of_list [] = empty" |
"set_of_list (x # xs) = insert x (set_of_list xs)"
```

lemma `"invar(set_of_list xs)"`
by`(induction xs)`
```
  (auto simp:  invar_empty invar_insert)
```
end

Implementing ADTs

1. Implement interface

©Mohammad Abdulaziz

Implementing ADTs

1. Implement interface
2. Prove specification

©Mohammad Abdulaziz

Implementing ADTs

1. Implement interface
2. Prove specification

Example

Binary search trees from the Week 4 sheet (Exercises 4.1–4.3):
functions `isin` and `ins` on type `'a tree`, abstraction function `tree_set`,
invariant `bst`.

©Mohammad Abdulaziz

Implementing ADTs

1. Implement interface
2. Prove specification

Example

Binary search trees from the Week 4 sheet (Exercises 4.1–4.3):
functions `isin` and `ins` on type `'a tree`, abstraction function `tree_set`, invariant `bst`.

Now implement locale `Set`:

©Mohammad Abdulaziz

In Isabelle: **interpretation**

©Mohammad Abdulaziz

In Isabelle: **interpretation**

```
interpretation Set
where empty = Leaf and isin = isin
  and insert = ins and set = tree_set
  and invar = bst
```

In Isabelle: **interpretation**

```
interpretation Set
where empty = Leaf and isin = isin
  and insert = ins and set = tree_set
  and invar = bst
proof
```

In Isabelle: **interpretation**

```
interpretation Set
where empty = Leaf and isin = isin
  and insert = ins and set = tree_set
  and invar = bst
proof
  show tree_set Leaf = {} ⟨proof⟩
```

In Isabelle: **interpretation**

```
interpretation Set
where empty = Leaf and isin = isin
  and insert = ins and set = tree_set
  and invar = bst
proof
  show tree_set Leaf = {} ⟨proof⟩
next
```

In Isabelle: **interpretation**

```
interpretation Set
where empty = Leaf and isin = isin
  and insert = ins and set = tree_set
  and invar = bst
proof
  show tree_set Leaf = {} ⟨proof⟩
next
  fix s assume bst s
  show tree_set (ins x s) = tree_set s ∪ {x}
  ⟨proof⟩
```

In Isabelle: **interpretation**

interpretation `Set`
where `empty = Leaf` **and** `isin = isin`
  and `insert = ins` **and** `set = tree_set`
  and `invar = bst`
proof
  show `tree_set Leaf = {}` ⟨proof⟩
next
  fix `s` **assume** `bst s`
  show `tree_set (ins x s) = tree_set s ∪ {x}`
  ⟨proof⟩
next
⋮
qed

In Isabelle: **interpretation**

interpretation `Set`
where `empty = Leaf` **and** `isin = isin`
  and `insert = ins` **and** `set = tree_set`
  and `invar = bst`
proof
  show `tree_set Leaf = {}` ⟨proof⟩
next
  fix `s` **assume** `bst s`
  show `tree_set (ins x s) = tree_set s ∪ {x}`
  ⟨proof⟩
next
⋮
qed

The cases are the Week 4 lemmas `tree_set_isin`, `tree_set_ins` and `bst_ins`.

Interpretation of `Set` also yields

- function `set_of_list :: 'a list ⇒ 'a tree`
- theorem `bst (set_of_list xs)`

©Mohammad Abdulaziz

Now back to search trees . . .

©Mohammad Abdulaziz